



OPERATING SYSTEMS UE24CS242B

Process Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:
 1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
 2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
 3. Some presentation transcripts from A. Frank – P. Weisberg
 4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau



OPERATING SYSTEMS



Process Concept

Pavan A C

Department of Computer Science

OPERATING SYSTEMS

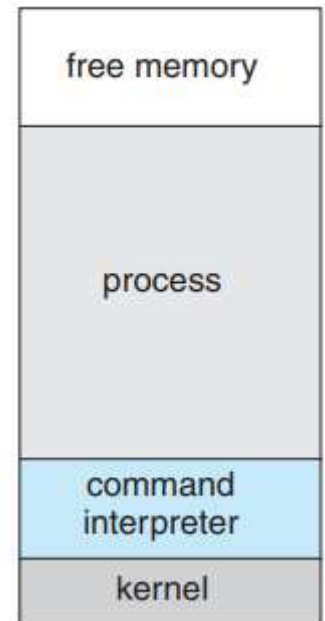
Process Concept

- An operating system executes a variety of programs:
 - Batch system – **jobs**
 - Time-shared systems – **user programs** or **tasks**
- Textbook uses the terms **job** and **process** almost interchangeably
- **Process** – a program in execution; process execution must progress in sequential fashion
- Program is **passive** entity stored on disk (**executable file**), process is **active**
 - Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be several processes
 - Consider multiple users executing the same program



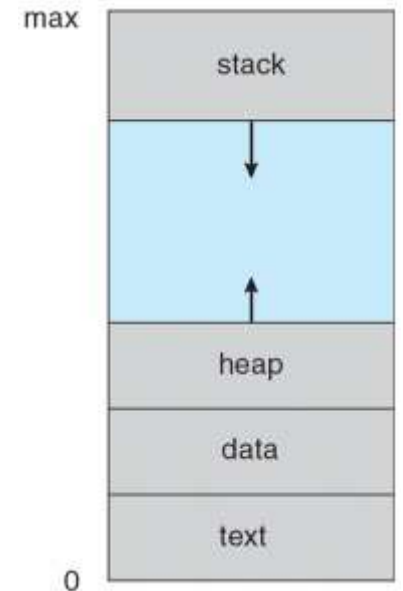
PES
UNIVERSITY

CELEBRATING 50 YEARS



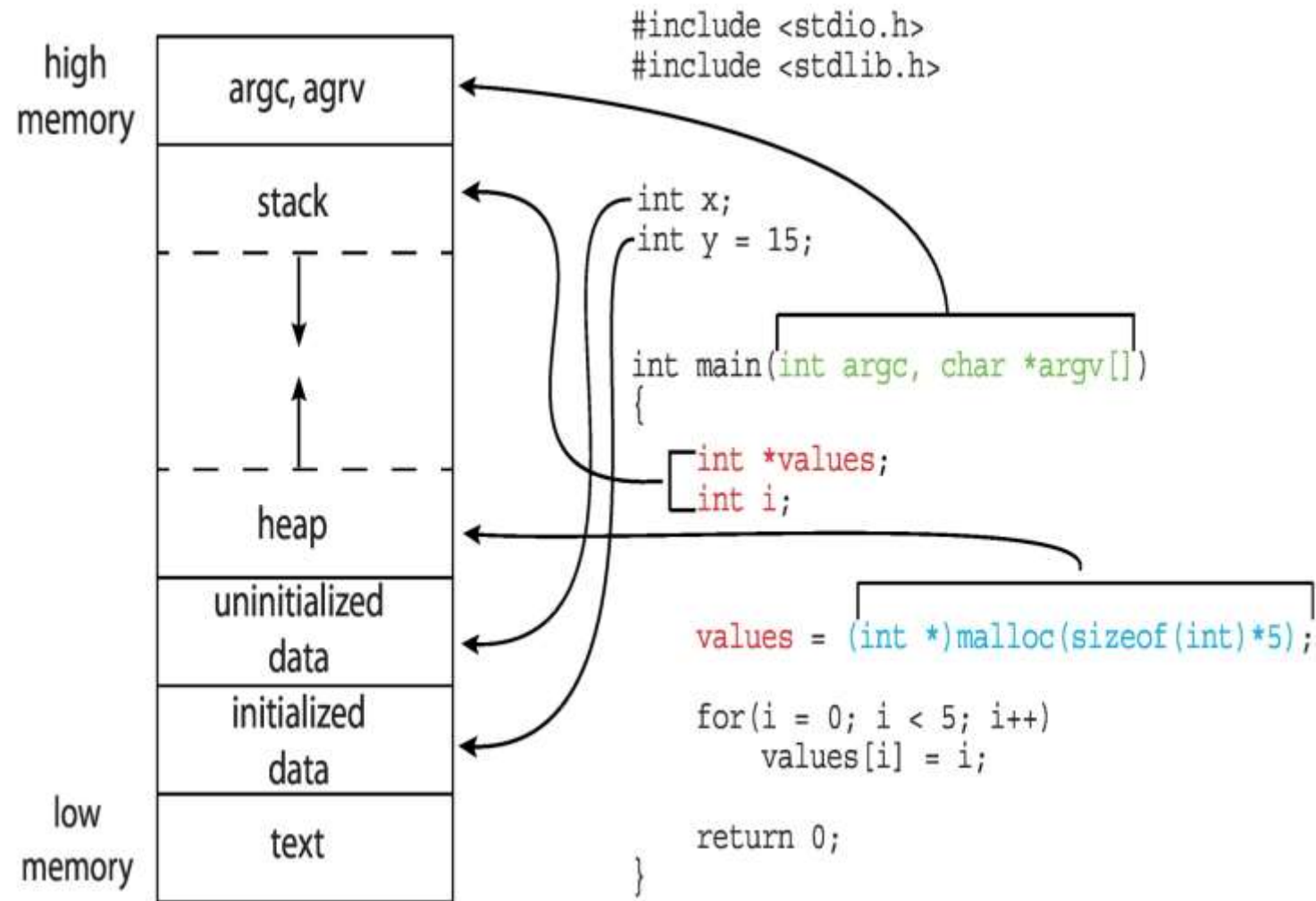
Structure of a process in memory

- The program code, also called **text section**.
 - Includes current activity including **program counter**, processor registers
- **Stack** containing temporary data
 - Function parameters, return addresses, local variables
- **Data section** containing global variables
- **Heap** containing memory dynamically allocated during run time



OPERATING SYSTEMS

Memory Layout of a C Program



PES
UNIVERSITY

CELEBRATING 50 YEARS

OPERATING SYSTEMS

Memory Layout of a process

```
suresh@PES1UG19CS553:~/OS$ gcc -o q1b.exe q1b.c
suresh@PES1UG19CS553:~/OS$ ./q1b.exe 5
Process id: 23059
```

```
suresh@PES1UG19CS553:~$ pmap 23059
23059:  ./q1b.exe 5
000055cefc914000      4K r-x-- q1b.exe
000055cefc914000      4K r---- q1b.exe
000055cefc914000      4K rw--- q1b.exe
000055cefc914000     132K rw--- [ anon ]
00007f26ef253000    1948K r-x-- libc-2.27.so
00007f26ef253000    2048K ----- libc-2.27.so
00007f26ef253000      16K r---- libc-2.27.so
00007f26ef253000       8K rw--- libc-2.27.so
00007f26ef253000      16K rw--- [ anon ]
00007f26ef253000     164K r-x-- ld-2.27.so
00007f26ef253000       8K rw--- [ anon ]
00007f26ef253000       4K r---- ld-2.27.so
00007f26ef253000       4K rw--- ld-2.27.so
00007f26ef253000       4K rw--- [ anon ]
00007f26ef253000     136K rw--- [ stack ]
00007f26ef253000      12K r---- [ anon ]
00007f26ef253000       8K r-x-- [ anon ]
ffffffffffffff600000      4K --x-- [ anon ]
total                4524K
```



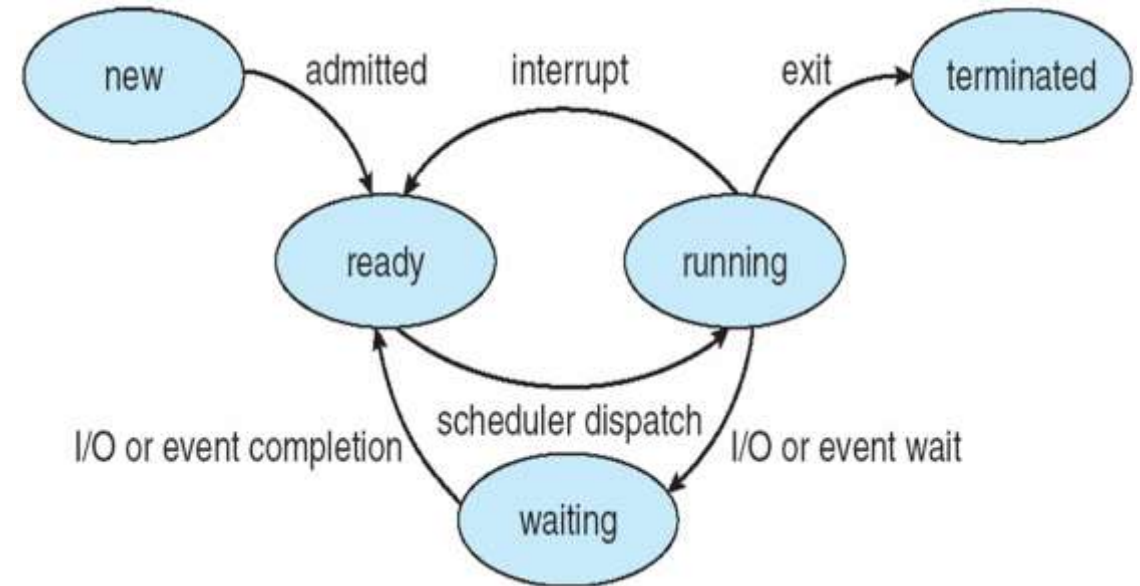
```
suresh@PES1UG19CS553:~$ cat /proc/23059/maps
55cefc914000-55cefc915000 r-xp 00000000 08:07 4338399 /home/suresh/OS/q1b.exe
55cefc915000-55cefc916000 r--p 00000000 08:07 4338399 /home/suresh/OS/q1b.exe
55cefc916000-55cefc917000 rw-p 00001000 08:07 4338399 /home/suresh/OS/q1b.exe
55cefe6e7000-55cefe708000 rw-p 00000000 00:00 0 [heap]
7f26ef253000-7f26ef43a000 r-xp 00000000 08:07 526643 /lib/x86_64-linux-gnu/libc-2.27.so
7f26ef43a000-7f26ef63a000 ---p 001e7000 08:07 526643 /lib/x86_64-linux-gnu/libc-2.27.so
7f26ef63a000-7f26ef63e000 r--p 001e7000 08:07 526643 /lib/x86_64-linux-gnu/libc-2.27.so
7f26ef63e000-7f26ef640000 rw-p 001eb000 08:07 526643 /lib/x86_64-linux-gnu/libc-2.27.so
7f26ef640000-7f26ef644000 rw-p 00000000 00:00 0
7f26ef644000-7f26ef66d000 r-xp 00000000 08:07 526515 /lib/x86_64-linux-gnu/ld-2.27.so
7f26ef66d000-7f26ef859000 rw-p 00000000 00:00 0
7f26ef859000-7f26ef86e000 r--p 00029000 08:07 526515 /lib/x86_64-linux-gnu/ld-2.27.so
7f26ef86e000-7f26ef86f000 rw-p 0002a000 08:07 526515 /lib/x86_64-linux-gnu/ld-2.27.so
7f26ef86f000-7f26ef870000 rw-p 00000000 00:00 0
7ffda2b15000-7ffda2b37000 rw-p 00000000 00:00 0 [stack]
7ffda2bae000-7ffda2bb1000 r--p 00000000 00:00 0 [vvar]
7ffda2bb1000-7ffda2bb3000 r-xp 00000000 00:00 0 [vdso]
ffffffffffff600000-ffffffffffff601000 --xp 00000000 00:00 0 [vsyscall]
suresh@PES1UG19CS553:~$
```


OPERATING SYSTEMS

Process State

As a process executes, it changes **state**

- **New**: The process is being created
- **Running**: Instructions are being executed
- **Waiting**: The process is waiting for some event to occur
- **Ready**: The process is waiting to be assigned to a processor
- **Terminated**: The process has finished execution



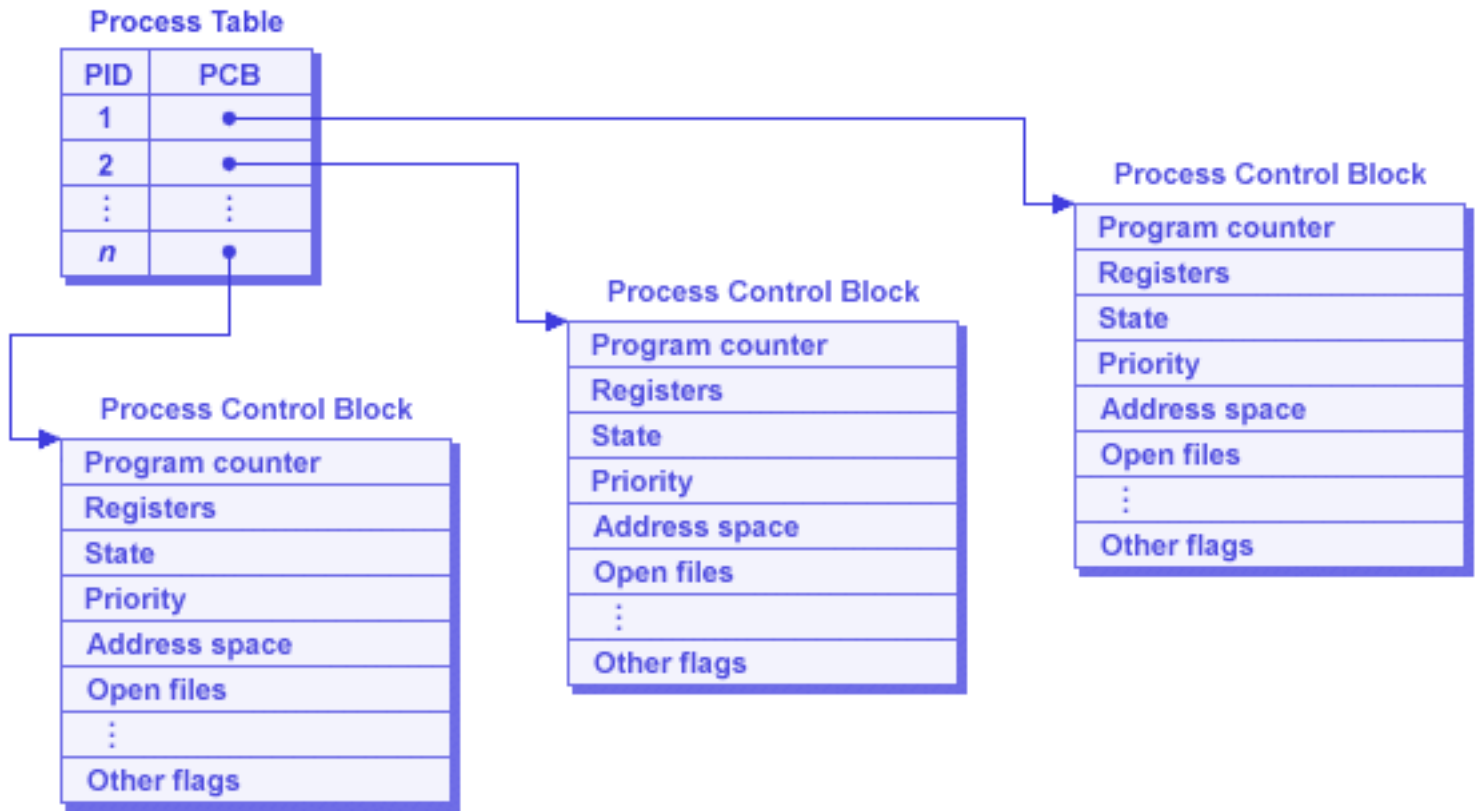
OPERATING SYSTEMS

Process Control Block (PCB)



PES
UNIVERSITY

CELEBRATING 50 YEARS



OPERATING SYSTEMS

Process Control Block (PCB)

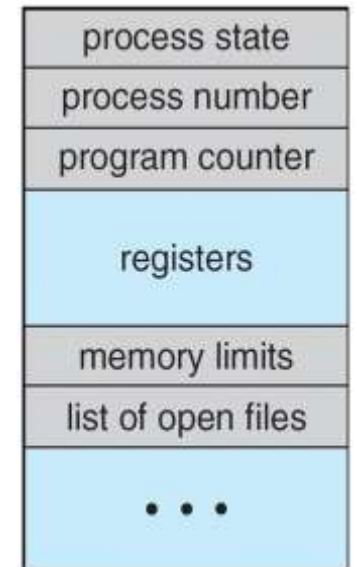
Each process is represented in the operating system by a Process Control Block (also called **task control block**)

- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files



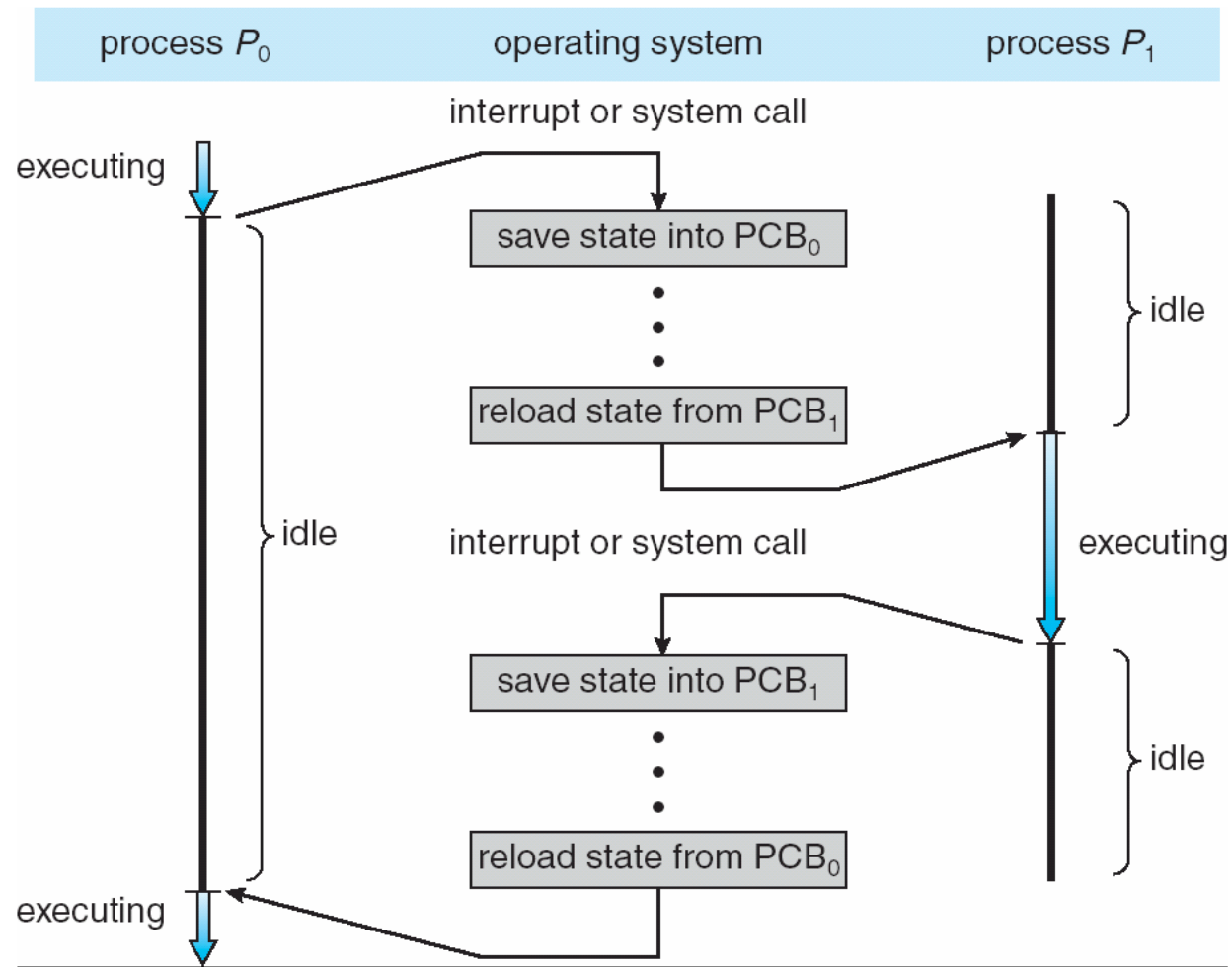
PES
UNIVERSITY

CELEBRATING 50 YEARS



OPERATING SYSTEMS

CPU switch from process to process



PES
UNIVERSITY

CELEBRATING 50 YEARS

OPERATING SYSTEMS

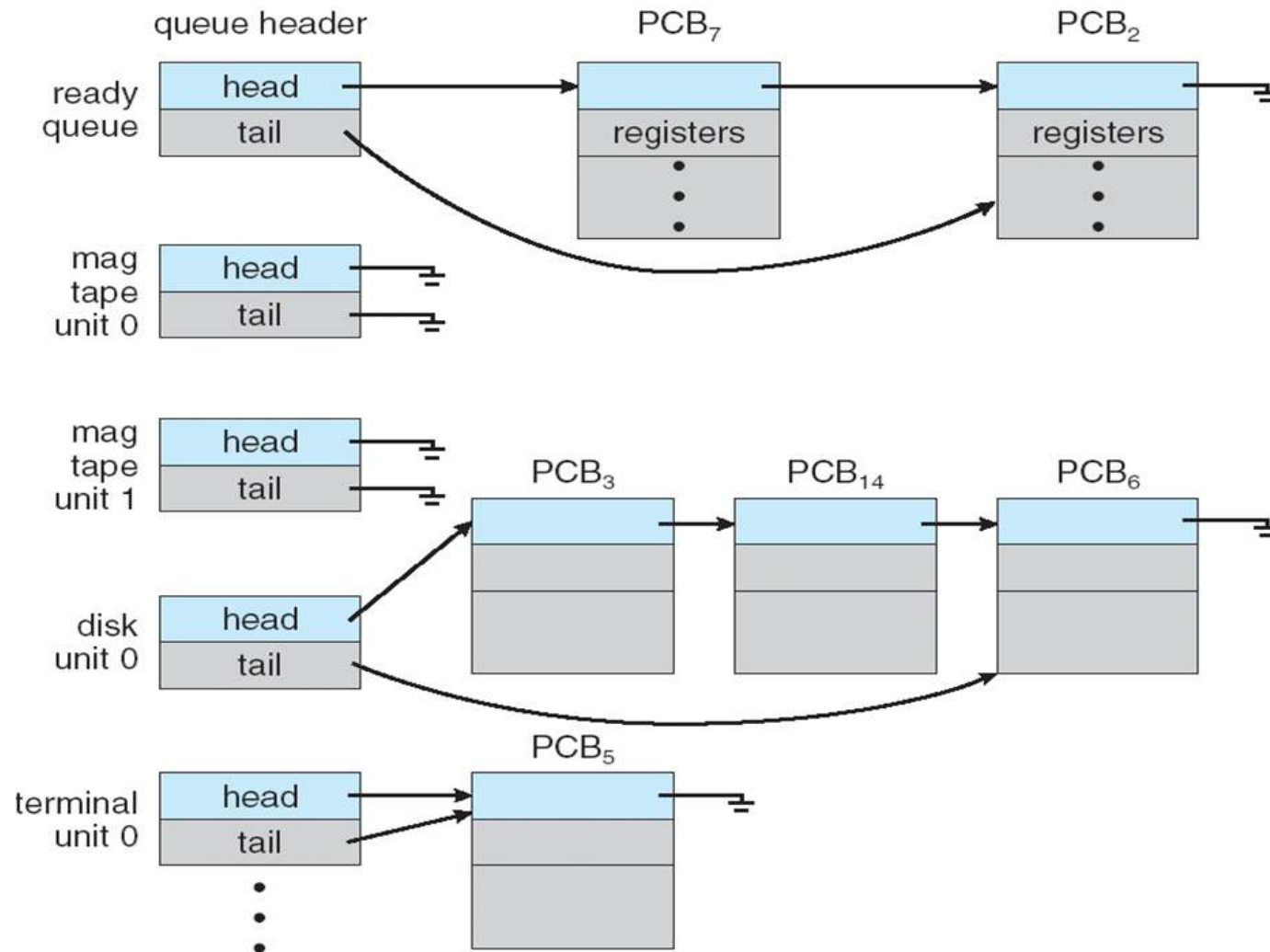
Process Scheduling

- Maximize CPU use, quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
 - **Job queue** – set of all processes in the system
 - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
 - **Device queues** – set of processes waiting for an I/O device
 - Processes migrate among the various queues



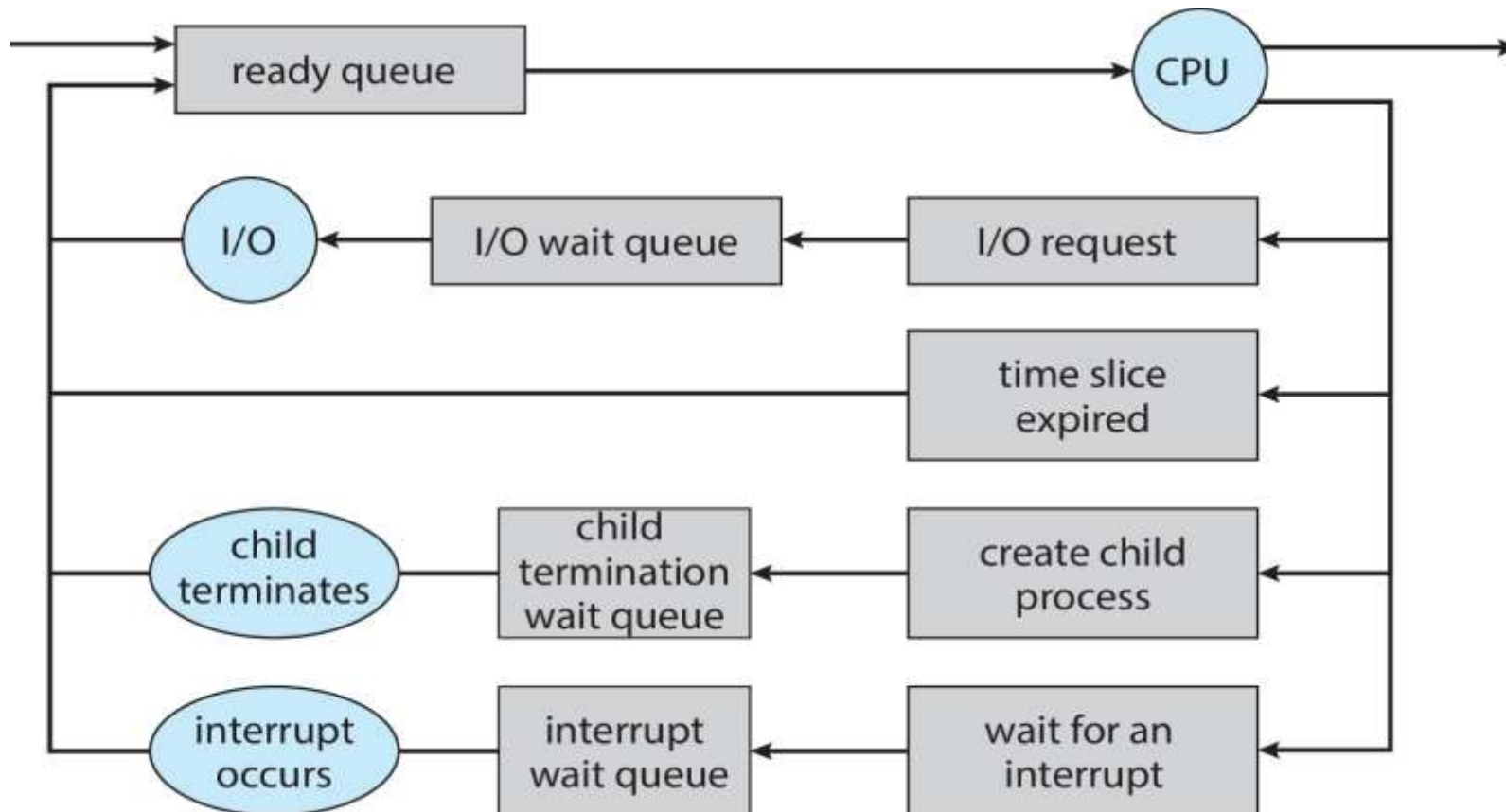
OPERATING SYSTEMS

Ready Queue And Various I/O Device Queues



OPERATING SYSTEMS

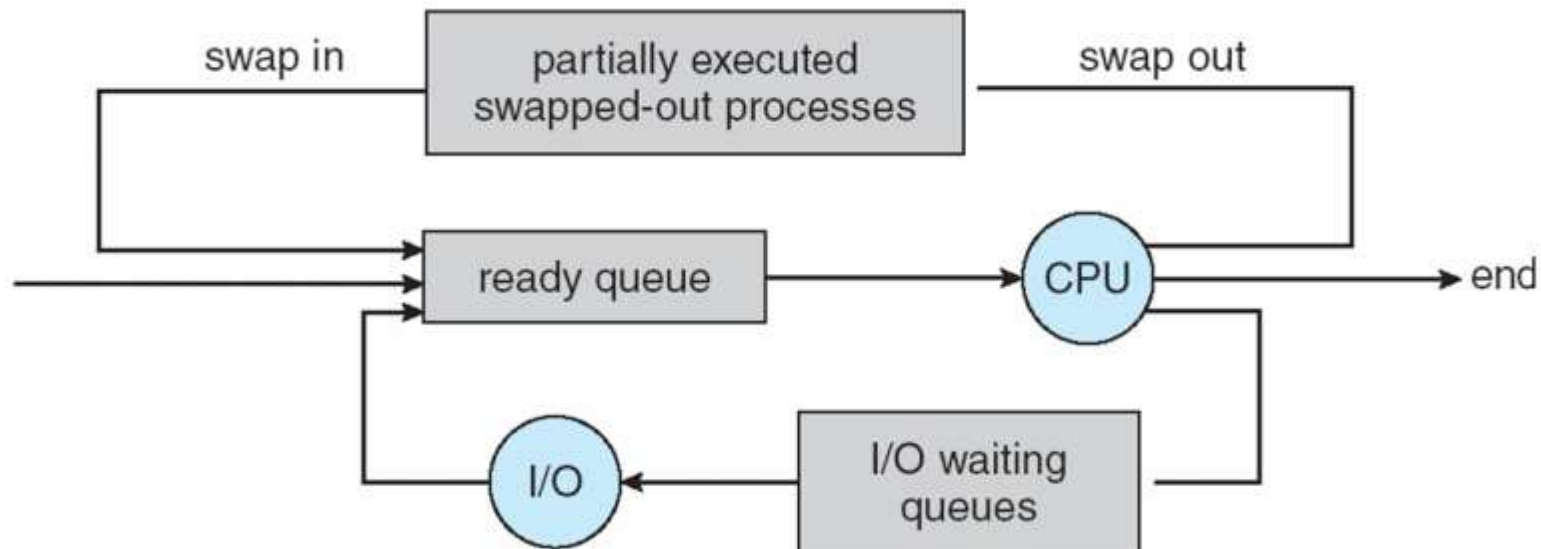
Representation of Process Scheduling



- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
 - Sometimes the only scheduler in a system
 - Short-term scheduler is invoked frequently (milliseconds) $\Rightarrow$ (must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
 - Long-term scheduler is invoked infrequently (seconds, minutes) $\Rightarrow$ (may be slow)
 - The long-term scheduler controls the **degree of multiprogramming**

- Processes can be described as either:
 - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
 - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good ***process mix***

- **Medium-term scheduler** can be added if degree of multiple programming needs to decrease
 - Remove process from memory, store on disk, bring back in from disk to continue execution: **swapping**



- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
 - The more complex the OS and the PCB → the longer the context switch
- Time dependent on hardware support
 - Some hardware provides multiple sets of registers per CPU → multiple contexts loaded at once

OPERATING SYSTEMS

Operations on Processes

System must provide mechanisms for:

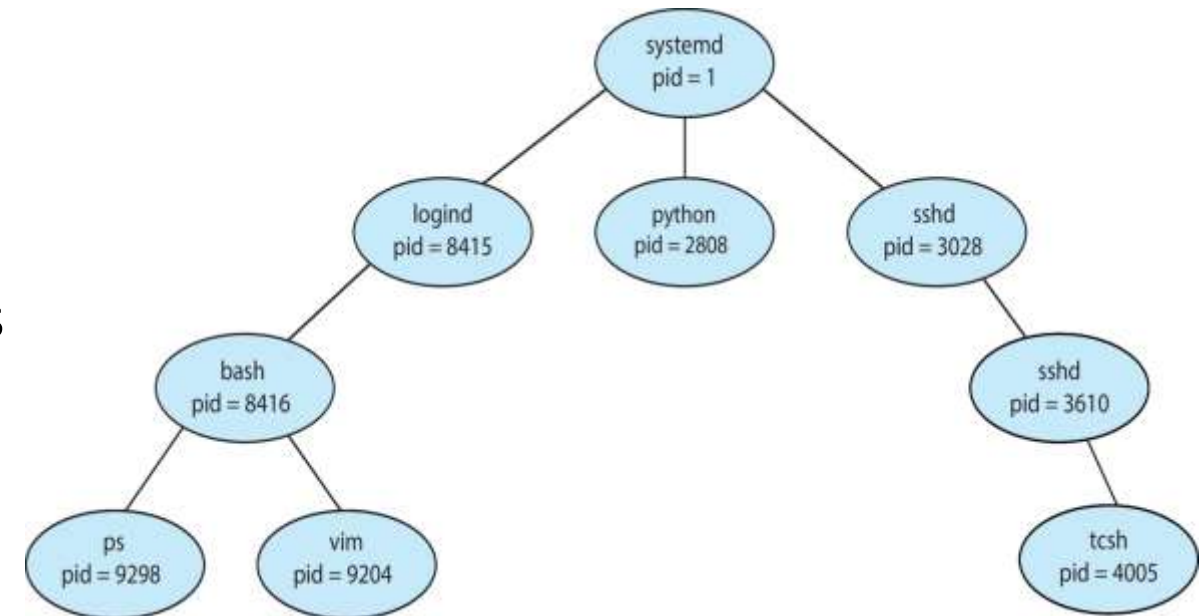
- process creation
- process termination



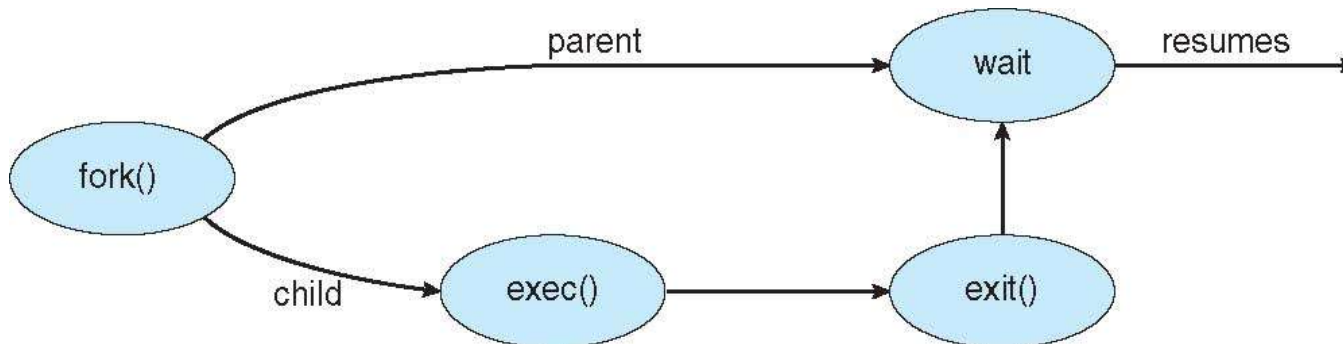
OPERATING SYSTEMS

Process Creation

- **Parent** process creates **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier (pid)**
- Resource sharing options
 - Parent and children share all resources
 - Children share subset of parent's resources
 - Parent and child share no resources
- Execution options
 - Parent and children execute concurrently
 - Parent waits until children terminate



- Address space
 - Child duplicate of parent
 - Child has a program loaded into it
- UNIX examples
 - **fork()** system call creates new process
 - **exec()** system call used after a **fork()** to replace the process' memory space with a new program



OPERATING SYSTEMS

C Program forking Separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```



- Process executes last statement and then asks the operating system to delete it using the **exit()** system call.
 - Returns status data from child to parent (via **wait()**)
 - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **abort()** system call. Some reasons for doing so:
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates

- Some operating systems do not allow child to exist if its parent has terminated. If a process terminates, then all its children must also be terminated.
 - **cascading termination.** All children, grandchildren, etc. are terminated.
 - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **wait()** system call. The call returns status information and the pid of the terminated process
 - **pid = wait(&status);**
- If no parent waiting (did not invoke **wait()**) process is a **zombie**
- If parent terminated without invoking **wait** , process is an **orphan**

OPERATING SYSTEMS

Process Termination

- What happens if parent process terminates before child process
- What happens if child process terminates before parent(i.e When the parent process in sleep)
- How to know the state of the process?





THANK YOU

Pavan A C

Department of Computer Science & Engineering

pavanac@pes.edu